MATERIAL DE EXAMEN & DEFENSA DE PROYECTO

SUZ CRM Comercial — Guía de Estudio Completa

Manual exhaustivo paso a paso para entender cada línea de código, su arquitectura y defender el proyecto ante un profesor o evaluador técnico.

ESTUDIANTE:	PROYECTO:	STACK TECNOLÓGICO:	ESTADO:
Desarrollador Junior	SUZ (Gestión Comercial)	React 18 • Node.js • MySQL	Producción • Verificado

ÍNDICE GENERAL DE LA GUÍA

1. Explicación General del Proyecto	Pág. 1	2. Estructura de Carpetas y Archivos	Pág. 2
3. Código Paso a Paso: Backend (API REST)	Pág. 3	4. Código Paso a Paso: Frontend (React & Redux)	Pág. 6
5. Recorrido Integral del Dato (Diagramas)	Pág. 10	6. Las 5 Partes Más Difíciles Explicadas	Pág. 12
7. Banco de Preguntas de Examen y Respuestas	Pág. 15	8. Guion Oral para la Presentación	Pág. 18
9. Glosario Técnico de Programación	Pág. 20	10. Plan de Estudio y Checklist de Control	Pág. 22

CAPÍTULO 1

Explicación General del Proyecto

1.1 ¿Qué hace la aplicación y cuál es su objetivo?

SUZ es un **CRM Comercial y de Fidelización de Ventas Recurrentes** diseñado a medida para una empresa distribuidora mayorista de materias primas e insumos (específicamente harina y azúcar).

El objetivo principal no es solo guardar clientes como una libreta de direcciones estática, sino **predecir el momento exacto en el que cada cliente se quedará sin mercadería** para que los preventistas y vendedores se anticipen y gestionen la recompra vía WhatsApp en un clic, además de controlar en vivo los saldos deudores de las cuentas corrientes.

1.2 ¿Qué problema real resuelve?

- **El problema de las ventas perdidas por olvido:** Las panaderías y comercios compran harina cada 7, 15 o 30 días. Si el vendedor no llama a tiempo, el cliente le compra a la competencia. El sistema resuelve esto calculando automáticamente la *Próxima Compra Estimada* y mostrando un **Semáforo Comercial** (Verde: con stock, Amarillo: compra hoy, Rojo: vencido).
- **El problema del cálculo mental de deudas:** En el rubro mayorista se venden miles de kilos con pagos parciales (seña o entrega de dinero). Los vendedores solían cometer errores calculando a mano. El sistema calcula en vivo: `Total = Kilos × Precio` y `Deuda = Total - Monto Pagado`.
- **El problema de la confidencialidad geográfica:** Vendedores de una provincia no deben ver ni editar clientes de otra provincia. El sistema implementó un control de acceso por roles (Login) que aísla las carteras de **Chaco (Gran Resistencia)**, **Corrientes** e **Interior de Chaco**.

1.3 ¿Cómo funciona desde que el usuario la abre? (Flujo de Usuario)

1. **Inicio de Sesión:** El usuario ingresa en `/login`. Si es vendedor de Chaco, Corrientes o Interior, ingresa su usuario y contraseña (o usa la botonera de 1 clic para la demo).
2. **Redirección de Seguridad:** Si el usuario es vendedor de Chaco, el guardián de rutas (`ProtectedRoute`) lo dirige exclusivamente a `/Chaco`. Si intenta tipear `/corrientes` en la URL, el sistema detecta que no tiene permiso y lo

devuelve a su zona. Si ingresa el `admin`, tiene acceso universal a todas las zonas.

3. **Tablero Provincial Segmentado:** La pantalla divide automáticamente los clientes en dos secciones visuales: 🔴 **Con Deuda Pendiente** y 🟢 **Clientes al Día**.
4. **Acción Comercial:** En cada tarjeta el vendedor ve el semáforo de reposición. Si está en amarillo o rojo, presiona el botón 💬 **WhatsApp**: se abre la aplicación con un mensaje prefabricado que incluye el nombre del comercio y del contacto para coordinar la reposición.
5. **Gestión de Cobranzas:** Si el cliente abona su saldo adeudado, el vendedor hace clic en 💵 **Saldar**. El sistema pone la deuda en $0 en tiempo real, actualiza la base de datos MySQL y dispara una notificación flotante **Toast** de confirmación.

1.4 Tecnologías Utilizadas y Justificación de Cada Una

TECNOLOGÍA	CAPA	¿POR QUÉ SE UTILIZÓ EN ESTE PROYECTO? (JUSTIFICACIÓN)
React 18	Frontend (UI)	Permite crear una Single Page Application (SPA) con componentes reactivos y modulares (Card, Form, HeaderNav). La interfaz se actualiza al instante sin recargar la página.
Redux & Redux-Thunk	Frontend (Estado)	Centraliza el estado global de los clientes en un único almacén (Store). Thunk permite despachar acciones asíncronas para comunicarse con la API mediante peticiones HTTP.
React Router v5	Frontend (Rutas)	Maneja la navegación interna en el navegador. Permite crear componentes de seguridad personalizados como `ProtectedRoute` para restringir vistas según el rol.
Node.js & Express	Backend (Servidor)	Entorno de ejecución liviano y framework minimalista para construir una API REST rápida basada en arquitectura Controlador-Ruta.
Sequelize ORM	Backend (Datos)	Mapeador Objeto-Relacional que abstrae SQL en código JavaScript. Permite sincronizar tablas con `sync({ alter: true })` sin borrar datos existentes.
MySQL 2	Base de Datos	Motor relacional robusto para garantizar integridad transaccional en datos comerciales, precios y deudas con tipos `DECIMAL(12, 2)`.
Dotenv (.env)	Seguridad	Separa la configuración sensible (contraseñas de base de datos y puertos) del código fuente, protegiendo las credenciales para que no se suban a GitHub.
UUID v4	Backend	Genera identificadores universales únicos de 36 caracteres para cada cliente, evitando colisiones numéricas secuenciales y mejorando la seguridad.

🎙️ **Tu Speech para el Profesor: ¿De qué se trata el proyecto?**

"El proyecto se llama SUZ y es un CRM comercial full-stack desarrollado con React, Redux, Node.js y MySQL. Nació de la necesidad real de una empresa distribuidora de harina y azúcar de transformar una libreta estática de clientes en una herramienta predictiva. El sistema analiza la fecha de última compra y la frecuencia habitual del cliente para alertar mediante un semáforo visual cuándo se le va a terminar el stock, facilitando contactarlo por WhatsApp en un clic. Además, calcula cuentas corrientes en vivo con montos de hasta diez mil millones de pesos y restringe la información por zonas geográficas mediante un sistema de login con roles."

CAPÍTULO 2

Estructura de Carpetas y Archivos

El proyecto está organizado en una arquitectura desacoplada Full Stack, dividida en dos aplicaciones independientes: `Server/` (Backend) y `my-app/` (Frontend).

```
Gestion-de-clientes/
├── .gitignore                  ← Blindaje de seguridad para no subir claves .env a Git
├── README.md                   ← Documentación general del repositorio
├── Server/                     ← BACKEND (API REST con Express y Sequelize)
│   ├── .env                    ← Variables privadas (puerto, clave de MySQL, CORS)
│   ├── .env.example            ← Plantilla pública de ejemplo para el equipo
│   ├── package.json            ← Dependencias del backend (express, sequelize, mysql2)
│   └── src/
│       ├── index.js            ← Punto de entrada: arranca servidor y sincroniza MySQL
│       ├── app.js              ← Configura middlewares (CORS dinámico, Morgan, JSON)
│       ├── db.js               ← Conexión Sequelize a MySQL y definición de relaciones
│       ├── crear_tablas.js     ← Script para migración manual segura con alter: true
│       ├── controller/
│       │   └── clientesController.js ← Lógica CRUD de clientes (findAll, create, update)
│       ├── models/
│       │   ├── Clientes.js     ← Modelo de datos de Clientes (columnas, tipos, UUID)
│       │   └── Pedidos.js      ← Modelo secundario de pedidos (scaffolding inicial)
│       └── routes/
│           ├── Index.js        ← Enrutador principal del servidor
│           ├── clientesRouter.js← Endpoints REST (/clientes: GET, POST, PUT, DELETE)
│           └── pedidosRouter.js ← Rutas de pedidos (scaffolding inicial)
└── my-app/                     ← FRONTEND (Single Page Application en React 18)
    ├── .env                    ← URL del backend (REACT_APP_API_URL=http://localhost:3001)
    ├── .env.example            ← Plantilla pública para el frontend
    ├── package.json            ← Dependencias frontend (react, redux, axios, router)
    └── src/
        ├── index.js            ← Montaje de React en el DOM con Providers (Redux y Toast)
        ├── index.css           ← Tokens CSS globales, Dark Mode y fuentes tipográficas
        ├── App.js              ← Router principal: barra superior y rutas protegidas
        ├── App.css             ← Estilos base del contenedor principal
        ├── context/
        │   ├── ToastContext.jsx ← Sistema de notificaciones flotantes con React Context
        │   └── Toast.css       ← Estilos Glassmorphism para las alertas Dark Mode
        ├── components/
        │   ├── ProtectedRoute.jsx ← Guardián de rutas que verifica login y rol zonal
        │   ├── HeaderNav/      ← Barra superior con usuario activo y botón de logout
        │   ├── Card/           ← Tarjeta CRM con semáforo comercial y botón WhatsApp
        │   └── SearchBar/      ← Barra de búsqueda reactiva por texto
        ├── redux/
        │   ├── store.js        ← Configuración de la tienda global con Thunk
        │   ├── actions.js      ← Acciones asíncronas HTTP consumiendo la API
        │   └── reducer.js      ← Reducer puro que actualiza el estado global
        ├── utils/
        │   ├── auth.js         ← Usuarios válidos, gestión de sesión en localStorage
        │   └── config.js       ← Centralización de la URL del backend (API_URL)
        └── views/
            ├── Login/          ← Pantalla de login con botones de prueba de 1 clic
            ├── Landing/        ← Portada general con selector de provincias (Admin)
            ├── Form/           ← Formulario con cálculos automáticos de deuda y kilos
            └── Home/           ← Vistas provinciales (Chaco, Corrientes, Interior)
```

2.1 Análisis de Impacto: ¿Qué pasaría si falta un archivo clave?

ARCHIVO	FUNCIÓN CRUCIAL	¿QUÉ PASARÍA SI FALTARA O ESTUVIERA DAÑADO?
`Server/src/db.js`	Crea la conexión Sequelize a MySQL y vincula los modelos.	El backend no puede arrancar. Arroja error de conexión inmediatamente y ninguna ruta responde porque no hay instancia de base de datos.
`Server/src/models/Clientes.js`	Define la estructura exacta de las columnas y tipos de datos en MySQL.	Sequelize no sabría cómo mapear los datos. Las consultas fallarían y la tabla `Clientes` no existiría en la base de datos.
`my-app/src/utils/config.js`	Centraliza la constante `API_URL` leída desde variables de entorno.	Todas las vistas y acciones de Redux perderían la dirección del servidor y no sabrían a qué puerto o dominio hacer las peticiones HTTP.
`my-app/src/components/ProtectedRoute.jsx`	Intercepta el acceso a cada pantalla y redirige si no hay permisos.	Cualquier persona que escriba la URL podría acceder a la información de cualquier provincia sin necesidad de iniciar sesión.
`my-app/src/redux/store.js`	Crea el contenedor del estado global de la aplicación.	El componente `<Provider>` en `index.js` fallaría y React arrojaría un error fatal en blanco al intentar montar la app.

2.2 Archivos Secundarios o No Utilizados en esta Etapa

Durante el análisis exhaustivo del repositorio se detectaron dos archivos que corresponden a un *scaffolding* inicial (estructura base de prueba) y que actualmente no intervienen en el flujo comercial principal:

- `Server/src/models/Pedidos.js`: Define un modelo básico de pedidos con solo `name`. En la arquitectura actual, los pedidos se registran y desglosan directamente dentro de cada registro de cliente (kilos, precio y producto).
- `Server/src/routes/pedidosRouter.js`: Contiene una ruta de prueba que responde el texto `"Estoy en user"`. No es consumida por el frontend de SUZ.

CAPÍTULO 3

Explicación del Código Paso a Paso: Backend (Node.js & Express)

El backend sigue una arquitectura modular en tres capas: **Ruta → Controlador → Modelo (ORM)**.

3.1 Conexión a Base de Datos con Sequelize: `Server/src/db.js`

> 💡 **Concepto Previo: ¿Qué es un ORM y por qué usamos Sequelize?**
>
> Un ORM (Object-Relational Mapping) es un traductor. En lugar de escribir código SQL manual como `SELECT * FROM Clientes WHERE id = 1`, usamos métodos de JavaScript como `Clientes.findByPk(1)`. Sequelize se encarga de hablar con MySQL por nosotros.

```
require('dotenv').config(); // 1. Carga variables desde el archivo .env const { Sequelize } =
require('sequelize'); const Clientesmodels = require("./models/Clientes"); // 2. Leemos credenciales con
valores por defecto seguros para desarrollo local const DB_NAME = process.env.DB_NAME ||
'gestionclientes'; const DB_USER = process.env.DB_USER || 'root'; const DB_PASSWORD =
process.env.DB_PASSWORD !== undefined ? process.env.DB_PASSWORD : 'gala12345'; const DB_HOST =
process.env.DB_HOST || 'localhost'; const DB_PORT = process.env.DB_PORT || 3306; // 3. Instanciamos la
conexión a MySQL const sequelize = new Sequelize(DB_NAME, DB_USER, DB_PASSWORD, { host: DB_HOST, port:
DB_PORT, dialect: 'mysql', logging: false, // Apaga los logs extensos de SQL en la consola });
Clientesmodels(sequelize); // 4. Inicializa el modelo pasándole la conexión
```

Explicación línea por línea:

- `require('dotenv').config()`: Lee el archivo privado `.env` y carga sus valores en el objeto global `process.env` de Node.js.
- `process.env.DB_PASSWORD ≠= undefined ? ... : 'gala12345'`: Es un operador ternario de respaldo. Si la variable existe en `.env` la utiliza; si por alguna razón el archivo `.env` falta, usa la clave por defecto para que la aplicación no se caiga.
- `dialect: 'mysql'`: Le indica a Sequelize que el motor de base de datos subyacente es MySQL (y no PostgreSQL o SQLite).

3.2 Modelo de Datos: `Server/src/models/Clientes.js`

> 💡 **Concepto Previo: ¿Por qué UUID en lugar de ID numérico 1, 2, 3?**
>
> Si usamos IDs numéricos secuenciales (1, 2, 3...), un atacante o competidor puede adivinar fácilmente cuántos clientes tiene la empresa o intentar verlos cambiando el número en la URL (`/clientes/4`). Un **UUID** (Universal Unique Identifier) genera una cadena alfanumérica única en el mundo de 36 caracteres (ej: `b372f107-074d-4abf-816e-a7534ccf0f33`), garantizando seguridad e imposibilidad de adivinarlo.

```
sequelize.define("Clientes", { id: { type: DataTypes.STRING(36), defaultValue: () ⇒ uuidv4(), // Genera automáticamente el UUID v4 primaryKey: true, allowNull: false, }, name: { type: DataTypes.STRING, unique: true }, contacto: { type: DataTypes.STRING, allowNull: true }, provincia: { type: DataTypes.STRING, allowNull: false }, frecuencia: { type: DataTypes.INTEGER, defaultValue: 7 }, // Días de reposición cantidadKg: { type: DataTypes.DECIMAL(10, 2), defaultValue: 0 }, precioKg: { type: DataTypes.DECIMAL(10, 2), defaultValue: 0 }, monto: { type: DataTypes.DECIMAL(12, 2), defaultValue: 0 }, // Pago entregado deuda: { type: DataTypes.DECIMAL(12, 2), defaultValue: 0 }, // Saldo pendiente });
```

Por qué usamos `DECIMAL(12, 2)` en lugar de `DECIMAL(8, 2)`:

- Un `DECIMAL(8, 2)` solo permite 6 dígitos enteros y 2 decimales, lo que equivale a un máximo de **$999.999,99** (menos de un millón de pesos). En la venta mayorista de harina por toneladas, un pedido real de un cliente ("La Reina") superaba $1.320.000, lo que hacía crashear la base de datos con el error *"Out of range value"*.
- Con `DECIMAL(12, 2)` el sistema soporta valores de hasta **$9.999.999.999,99** (casi diez mil millones de pesos), ideal para el comercio mayorista argentino con alta inflación.

3.3 Arranque y Migración Segura: `Server/src/index.js`

```
const app = require("./app"); const { sequelize } = require("./db"); const PORT = process.env.PORT || 3001; (async () ⇒ { try { await sequelize.authenticate(); // 1. Prueba que el usuario y clave de MySQL sean válidos await sequelize.sync({ alter: true }); // 2. MIGRADOR SEGURO: actualiza columnas sin borrar datos app.listen(PORT, () ⇒ { console.log(`🚀 Servidor corriendo en http://localhost:${PORT}`); }); } catch (error) { console.error("❌ Error:", error.message); process.exit(1); } })();
```

> ⚠️ **Pregunta Clave de Examen: ¿Por qué nunca usamos force: true en producción?**
>
> Si pusiéramos `sync({ force: true })`, Sequelize ejecutaría un comando `DROP TABLE` destructivo en cada reinicio del servidor, borrando absolutamente todos los clientes y las ventas guardadas. En su lugar, usamos `{ alter: true }`, que compara los modelos en JavaScript contra la estructura existente de MySQL y ejecuta sentencias `ALTER TABLE ADD COLUMN`, preservando el 100% de los datos históricos.

3.4 Controlador de Clientes: `Server/src/controller/clientesController.js`

El controlador es el cerebro del backend: recibe la información del frontend, la valida, interactúa con el modelo y responde con código de estado HTTP.

```
// CREAR CLIENTE (POST /clientes) const nuevoClientes = async (req, res) => { try { // 1. Desestructuración de los datos que viajan en el cuerpo (req.body) const { name, contacto, provincia, cantidadKg, precioKg, monto, deuda, frecuencia } = req.body; // 2. Conversión segura a tipos numéricos para no guardar NaN en MySQL const newClient = await Clientes.create({ name, contacto: contacto || null, provincia, frecuencia: frecuencia ? Number(frecuencia) : 7, cantidadKg: cantidadKg ? Number(cantidadKg) : 0, precioKg: precioKg ? Number(precioKg) : 0, monto: monto ? Number(monto) : 0, deuda: deuda ? Number(deuda) : 0, }); // 3. Respuesta HTTP 200 con el objeto creado en formato JSON res.status(200).json({ message: "Cliente registrado exitosamente", cliente: newClient }); } catch (error) { res.status(500).json({ error: error.message }); } };
```

CAPÍTULO 4

Explicación del Código Paso a Paso: Frontend (React & Redux)

4.1 Control de Acceso y Guardián de Rutas: `auth.js` y `ProtectedRoute.jsx`

Concepto Previo: ¿Cómo sabe React si el usuario está logueado?

El protocolo HTTP es "stateless" (no tiene memoria entre páginas). Para recordar la sesión en el navegador sin recargar, guardamos un objeto de usuario en el `localStorage` del navegador. Al navegar, nuestro componente de seguridad lee el `localStorage` y decide si renderiza la pantalla o redirige.

```
// my-app/src/components/ProtectedRoute.jsx const ProtectedRoute = ({ component: Component, render, rolPermitido, ...restoDeProps }) => { return ( <Route {...restoDeProps} render={(props) => { const usuario = obtenerUsuarioActual(); // Lee desde localStorage // 1. Si no hay sesión activa, va directo al login if (!usuario) return <Redirect to="/login" />; // 2. El Administrador tiene acceso total a cualquier ruta if (usuario.rol === "admin") return Component ? <Component {...props} /> : render(props); // 3. Si coincide el rol (ej: rol 'chaco' en pantalla /Chaco) if (usuario.rol === rolPermitido) return Component ? <Component {...props} /> : render(props); // 4. SEGURIDAD: Si un vendedor intenta entrar a una provincia ajena, lo rebotamos a la suya const mapaRutas = { chaco: "/Chaco", corrientes: "/corrientes", interior: "/Interior" }; return <Redirect to={mapaRutas[usuario.rol] || "/login"} />; }} /> ); };
```

4.2 Motor del Semáforo Comercial y Ciclo de Recompra: `Card.js`

Esta es la función matemática que convierte a la aplicación en un CRM inteligente:

```
export const calcularCicloComercial = (fechaUltima, diasFrecuencia = 7) => { if (!fechaUltima) return { badgeTexto: "⚪ Sin fecha", estado: "sin-fecha" }; const dias = Number(diasFrecuencia) || 7; const ultima = new Date(fechaUltima); // Fecha Próxima = Última Compra + Días de Frecuencia habitual const proxima = new Date(ultima); proxima.setDate(proxima.getDate() + dias); const hoy = new Date(); hoy.setHours(0, 0, 0, 0); // Diferencia en milisegundos convertida a días enteros const diffDias = Math.round((proxima.getTime() - hoy.getTime()) / (1000 * 60 * 60 * 24)); if (diffDias < 0) { return { badgeTexto: `🔴 Vencido (${Math.abs(diffDias)} días)`, estado: "vencido" }; } else if (diffDias <= 2) { return { badgeTexto: "🟡 ¡Toca comprar HOY!", estado: "hoy" }; } return { badgeTexto: `🟢 En fecha (${diffDias} días restantes)`, estado: "en-fecha" }; };
```

4.3 Conexión Asíncrona con el Backend: `my-app/src/redux/actions.js`

Concepto Previo: ¿Qué es una acción asíncrona en Redux Thunk?

Por defecto, las acciones de Redux son instantáneas y síncronas (retornan un simple objeto `{ type, payload }`). Como comunicarnos con el servidor MySQL toma tiempo por la red, **Redux Thunk** nos permite retornar una función asíncrona que hace un `await axios.get(...)` y recién cuando llegan los datos del servidor, despacha el resultado al reducer.

```
import axios from "axios"; import { API_URL } from "../utils/config"; // Viene de REACT_APP_API_URL export const postClientes = (payload) => { return async function (dispatch) { try { // 1. Envía los datos del nuevo cliente por HTTP POST a Express const res = await axios.post(`${API_URL}/clientes`, payload); // 2. Despacha al reducer para que la pantalla se actualice al instante dispatch({ type: "POST_CLIENTE", payload: res.data }); return res; } catch (error) { console.error("❌ Error al crear cliente:", error); } }; };
```

4.4 Sistema de Notificaciones Toast: `ToastContext.jsx`

Reemplazó a los `alert()` nativos. Usa el hook `useToast()` y React Context:

```
const { showToast } = useToast(); // En lugar de: alert("Cliente registrado exitosamente") showToast("¡Cliente registrado exitosamente!", "success"); showToast("El cliente ya existe en esta provincia", "error");
```

CAPÍTULO 5

Recorrido Integral del Dato (Flujo de la Información)

A continuación se ilustra qué camino sigue exactamente la información desde que el usuario completa el formulario hasta que se almacena físicamente en el disco duro dentro de MySQL:

```
1. USUARIO (Pantalla Form.jsx)
   | Ingresa: Razón Social, 500 kg de harina a $850/kg, pago de $300.000.
   | React calcula en vivo: Total = $425.000 | Deuda = $125.000.
   ▼
2. DISPATCH (Redux Thunk en actions.js)
   | dispatch(postClientes(clienteAEnviar))
   | Axios emite una petición HTTP: POST http://localhost:3001/clientes
   ▼
3. EXPRESS BACKEND (Server/src/app.js)
   | Middleware cors() verifica el origen permitido (CLIENT_URL).
   | Middleware express.json() transforma el texto plano en un objeto JavaScript.
   | clientesRouter.js deriva la petición hacia nuevoClientes().
   ▼
4. CONTROLADOR (clientesController.js)
   | Desestructura req.body, valida números y llama a Clientes.create().
   ▼
5. SEQUELIZE ORM & MYSQL2 DRIVER
   | Traduce la orden a SQL: INSERT INTO Clientes (id, name, monto, deuda...) VALUES (...)
   ▼
6. BASE DE DATOS (MySQL en puerto 3306)
   | Guarda el registro en la tabla con ID UUID v4. Responde "OK".
   ▼
7. VUELTA AL FRONTEND & EXPERIENCIA DE USUARIO
   | Express responde con status 200 y el JSON del cliente.
   | Redux almacena el cliente en el Store.
   | ToastContext dispara la notificación animada verde "¡Cliente registrado exitosamente!".
   | history.push("/Chaco") redirige al vendedor a la vista de su provincia.
   | Chaco.jsx filtra Number(c.deuda) > 0 y ubica al cliente en la sección roja "Con Deuda".
```

CAPÍTULO 6

Las 5 Partes Más Difíciles del Proyecto (Explicadas para Principiantes)

1. ¿Por qué usamos Redux y Redux-Thunk en lugar de un simple `useState`?

> **Explicación Simple:**
> Imagina que `useState` es una nota adhesiva que solo puede leer una persona en su escritorio (un componente). Si el formulario crea un cliente, la vista de Chaco no se entera porque está en otra habitación. **Redux es una pizarra gigante en el pasillo (el Store)**: cualquier componente puede escribir en la pizarra y todos los demás componentes se actualizan automáticamente al instante.

En nuestro código: Cuando `Form.jsx` registra un cliente, despacha `POST_CLIENTE`. El Store se actualiza y cuando el usuario entra a `Chaco.jsx`, el nuevo cliente ya aparece listado sin tener que recargar el navegador ni pasar datos de padre a hijo por 10 niveles (evitando el llamado *Prop Drilling*).

2. La Migración Segura en MySQL: `sync({ alter: true })` vs `sync({ force: true })`

> **Explicación Simple:**
> Imagina que construiste una casa de dos habitaciones (tu tabla en la base de datos) y ya viven personas adentro (los datos de los clientes). Si quieres agregar un garaje:
> - `force: true` : Demuele la casa con una bola de demolición (`DROP TABLE`) y construye una casa nueva vacía. Toda la gente muere (se pierden los datos).
> - `alter: true` : Llama a los albañiles para que construyan únicamente el garaje (`ALTER TABLE ADD COLUMN`) sin molestar a los que viven adentro.

3. El Guardián de Rutas (Higher Order Component en React Router v5)

> **Explicación Simple:**
> En React Router estándar, una ruta es pública: quien tipea `/Chaco` en el navegador entra sin filtro. `ProtectedRoute` actúa como el guardia de seguridad de un edificio: antes de dejarte pasar a la oficina, te pide identificación (lee el `localStorage`). Si no tienes gafete, te manda a la recepción (`/login`). Y si tienes gafete de vendedor de Chaco pero intentas entrar a la oficina de Corrientes, te acompaña de vuelta a tu oficina propia.

4. Inmutabilidad en Redux y el Operador Spread (`...state`)

> **Explicación Simple:**
> En JavaScript, React detecta que debe redibujar la pantalla cuando cambia la *referencia de memoria* de un objeto. Si haces `state.clientes.push(nuevo)` , modificas el array original, React no nota el cambio y la pantalla se queda congelada. Al hacer `[...state.clientes, action.payload]` , creas una fotocopia nueva con el nuevo elemento agregado. React ve una nueva dirección de memoria y redibuja la pantalla inmediatamente.

5. Normalización y Sincronización de Nombres Full Stack

> **Explicación Simple:**
> Uno de los errores más comunes en proyectos junior es que la pantalla llama a un campo `frecuencia` , el controlador intenta leer `frecuenciaCompra` y el modelo de MySQL lo espera como `FrecuenciaDecompra` . Como JavaScript distingue mayúsculas y minúsculas (es *case-sensitive*), MySQL recibía `undefined` y guardaba ceros o nulos. Lo solucionamos estandarizando exactamente las mismas variables en las 3 capas: `contacto` , `frecuencia` , `producto` , `marca` , `cantidadKg` , `precioKg` , `monto` y `deuda` .

CAPÍTULO 7

Banco de Preguntas de Examen y Respuestas Modelo

A continuación tienes las preguntas reales que cualquier profesor o jurado técnico te hará, con la respuesta exacta que debes dar:

❓ Pregunta 1: "¿Por qué utilizaste Redux si React ya tiene useState y Context API?"

Respuesta modelo: "Utilicé Redux porque el estado de los clientes es compartido por múltiples pantallas que no tienen relación directa de padre e hijo: el formulario de alta, las tres vistas provinciales y las búsquedas por nombre. Con Redux logramos una arquitectura unidireccional y un único punto de verdad (Single Source of Truth). Además, al combinarlo con Redux Thunk, podemos encapsular todas las llamadas asíncronas hacia la API de Express de forma ordenada y predecible fuera de los componentes de la interfaz."

❓ Pregunta 2: "¿Cómo manejaste la persistencia de sesión en el frontend sin usar JWT en el backend?"

Respuesta modelo: "Para esta etapa del proyecto implementamos una solución liviana basada en roles utilizando `localStorage` del navegador mediante el módulo `auth.js`. Cuando el usuario se autentica en la vista de Login, guardamos su objeto de perfil (rol y zona). Nuestro componente `ProtectedRoute` intercepta cada cambio de ruta en React Router v5: si el usuario no existe en `localStorage`, redirige a `/login`; y si el rol no coincide con la provincia a la que intenta ingresar, lo redirige automáticamente a su área asignada. Para producción, el siguiente paso evolutivo sería migrar esta validación a tokens firmados JWT con cookies HttpOnly."

❓ Pregunta 3: "¿Cómo resolviste el desbordamiento numérico en los campos de deuda en MySQL?"

Respuesta modelo: "Originalmente los campos de monto y deuda estaban tipados como `DECIMAL(8, 2)`, lo que imponía un límite máximo de $999.999,99. Al cargar pedidos reales de clientes mayoristas con deudas superiores a 1,3 millones de pesos, MySQL lanzaba el error 'Out of range value'. Lo solucioné modificando el modelo en Sequelize a `DECIMAL(12, 2)`, lo que nos permite almacenar importes comerciales de hasta diez mil millones de pesos con precisión de centavos sin redondeos erróneos de coma flotante."

❓ Pregunta 4: "¿Qué es CORS y por qué tuviste que configurarlo en app.js?"

Respuesta modelo: "CORS significa Cross-Origin Resource Sharing (Intercambio de Recursos de Origen Cruzado). Es un mecanismo de seguridad de los navegadores que bloquea las peticiones cuando el frontend (que corre en el puerto 3000) intenta pedirle datos a un servidor en otro puerto o dominio (el backend en el puerto 3001). En `app.js` configuramos el middleware `cors()` leyendo la variable de entorno `CLIENT_URL` para indicarle al servidor qué dominios tienen autorización para consumir nuestros endpoints."

❓ Pregunta 5: "¿Cómo funciona el botón Saldar en la tarjeta?"

Respuesta modelo: "En `Card.js`, el botón Saldar toma el monto total del pedido (kilos × precio) y ejecuta un dispatch de `updateClientes` enviando el cliente con `monto: montoTotal` y `deuda: 0`. Esto viaja por Axios por método PUT a `/clientes/:id`. El controlador de Express actualiza la fila en MySQL con `cliente.save()` y el frontend actualiza el estado local mostrando al instante la deuda en verde ($0) y disparando una notificación Toast de confirmación sin recargar la pantalla."

CAPÍTULO 8

Guion Oral para Presentar el Proyecto (Paso a Paso)

Este guion está escrito con un tono natural y seguro para que expongas el proyecto en 4 a 5 minutos frente al profesor:

🎙️ 1. Introducción (30 segundos)

"Buenos días, profesor. Hoy les presento **SUZ**, un CRM comercial y de reposición predictiva diseñado para distribuidores mayoristas de alimentos como harina y azúcar. El proyecto surgió para resolver un problema crítico del rubro: la pérdida de ventas recurrentes porque los preventistas no sabían con certeza qué día exacto cada cliente se quedaba sin mercadería, sumado a las dificultades para controlar deudas y saldos en cuenta corriente."

🎙️ 2. Arquitectura y Tecnologías (45 segundos)

"Desarrollé el proyecto en una arquitectura desacoplada full-stack. En el frontend utilicé **React 18** con **Redux y Redux-Thunk** para el manejo del estado global, y **React Router v5** para la navegación. El backend está construido sobre **Node.js y Express**, y se conecta mediante el ORM **Sequelize** a una base de datos relacional **MySQL**. Además, implementé un desacople total de credenciales y variables de entorno mediante `dotenv` y un archivo de configuración centralizado tanto en cliente como en servidor."

🎙️ 3. Demostración Práctica: Login y Control de Acceso (60 segundos)

"Al ingresar a la aplicación vemos la pantalla de inicio de sesión con estética Dark Mode. La aplicación cuenta con cuatro perfiles de acceso: el Administrador y los vendedores asignados a **Chaco (Gran Resistencia)**, **Corrientes** e **Interior de Chaco**. Si inicio sesión como vendedor de Chaco, el guardián de rutas `ProtectedRoute` me dirige directamente a mi zona de trabajo y me impide ver o modificar clientes de otras provincias, garantizando la privacidad comercial. El administrador, por su parte, tiene una botonera en la barra superior para auditar y cambiar entre todas las áreas."

🎙️ 4. El Corazón del CRM: Semáforo y Cuenta Corriente (60 segundos)

"Dentro de cada provincia, las tarjetas inteligentes de clientes están divididas en dos grupos: con deuda y al día. Lo más importante es el **Semáforo Comercial**: el sistema toma la fecha de última compra y le suma la frecuencia habitual del cliente (por ejemplo, cada 7 días). Si faltan días para reponer aparece en verde; si es momento de contactarlo aparece en amarillo alertando '¡Toca comprar hoy!'; y si la fecha venció aparece en rojo. Desde la misma tarjeta el vendedor hace clic en el botón de WhatsApp y se abre la conversación con un mensaje automático personalizado. Por otro lado, la cuenta corriente multiplica automáticamente kilos por precio unitario y muestra la deuda restante. Si el cliente entrega el dinero adeudado, con un solo clic en 'Saldar' la deuda se va a $0 y se guarda en MySQL."

🎙️ 5. Desafíos Técnicos Resueltos y Cierre (45 segundos)

"Los mayores retos del proyecto fueron sincronizar los nombres de los atributos en las tres capas para que no se pierdan datos, migrar la base de datos de forma segura con `alter: true` sin borrar las filas reales, ampliar los campos a `DECIMAL(12, 2)` para permitir facturaciones de hasta diez mil millones de pesos, y reemplazar los alerts nativos del navegador por notificaciones Toast fluidas. El resultado es un sistema robusto, testeado y listo para producción. Quedo a disposición para cualquier consulta técnica sobre el código."

CAPÍTULO 9

Glosario Técnico de Programación (Términos Reales del Proyecto)

TÉRMINO	DEFINICIÓN SENCILLA	EJEMPLO EN EL PROYECTO SUZ
API REST	Conjunto de reglas y endpoints HTTP (GET, POST, PUT, DELETE) para comunicar el frontend con el backend.	`POST http://localhost:3001/clientes` para dar de alta un comercio en el servidor.
ORM (Sequelize)	Librería que mapea tablas relacionales a clases y objetos en JavaScript.	`Clientes.findByPk(id)` en lugar de `SELECT * FROM Clientes WHERE id = ?`.
Single Source of Truth	Principio arquitectónico donde todos los componentes leen los datos de un único almacén central.	El Store de Redux donde reside el array `clientes` para toda la aplicación.
Middleware	Función intermedia que intercepta una petición HTTP antes de que llegue al controlador final.	`express.json()` que parsea el cuerpo de la solicitud, o `cors()` que valida el origen.
Hook de React	Función especial (que empieza con `use`) que permite a componentes funcionales usar estado y efectos.	`useState` para formularios locales, `useEffect` para traer datos, `useToast` para alertas.
HOC / ProtectedRoute	Componente envoltorio que condiciona el renderizado de una vista a si se cumplen ciertas reglas lógicas.	`ProtectedRoute` verifica si hay sesión en `localStorage` antes de mostrar la provincia.
Spread Operator (`...`)	Sintaxis de 3 puntos que descompone un objeto o array para clonarlo sin mutar el original.	`{ ...state, clientes: action.payload }` en `reducer.js`.
Variables de Entorno	Valores de configuración externos al código cargados dinámicamente según el entorno (desarrollo o producción).	`process.env.DB_PASSWORD` en el servidor o `REACT_APP_API_URL` en React.
Glassmorphism	Técnica de diseño UI que imita un vidrio translúcido con desenfoque de fondo y bordes brillantes.	Las notificaciones flotantes en `Toast.css` con `backdrop-filter: blur(16px)`.

CAPÍTULO 10

Plan de Estudio y Checklist de Autoevaluación

10.1 Orden Recomendado de Estudio (Cronograma de 3 Fases)

1. **Fase 1: El Sentido de Negocio y la Demo (Día 1):** Practicar el recorrido completo en el navegador. Probar ingresar como admin y como vendedor de Chaco. Crear un cliente con deuda y saldarlo. Entender qué es el semáforo y por qué existe.
2. **Fase 2: El Camino del Dato (Día 2):** Estudiar el Capítulo 5 (Recorrido Integral del Dato). Poder explicar con el dedo cómo viaja la información desde el formulario de React hasta la tabla de MySQL.
3. **Fase 3: Las Preguntas Técnicas y el Guion (Día 3):** Leer en voz alta el Capítulo 8 (Guion Oral) hasta poder decirlo con fluidez sin leer el papel. Practicar las 5 preguntas del Capítulo 7.

10.2 Checklist de Autoevaluación (¿Realmente entendí el proyecto?)

Marca con una cruz cada punto solo cuando puedas explicarlo con tus propias palabras:

- [ ] Sé explicar por qué SUZ es un CRM predictivo y no solo un CRUD básico.
- [ ] Entiendo por qué usamos `alter: true` y qué desastre provocaría `force: true`.
- [ ] Sé por qué ampliamos el campo deuda a `DECIMAL(12, 2)`.
- [ ] Puedo explicar la fórmula matemática del semáforo comercial (última compra + frecuencia).
- [ ] Sé cómo funciona `ProtectedRoute` y qué verifica antes de dejar pasar a un vendedor.
- [ ] Puedo justificar por qué las claves van en `.env` y se agregan a `.gitignore`.

- [ ] Sé por qué "Interior" se aclaró a "Interior de Chaco" tanto en la UI como en el negocio.
- [ ] Conozco las credenciales de demo (admin, chaco, corrientes, interior con clave 123).
